Lecture: 2

Stacks

"Data dominates. If you've chosen the right data structures and organized things well, the algorithms will almost always be self-evident. Data structures, not algorithms, are central to programming."

Rob Pike

Stack:

Last In First Out

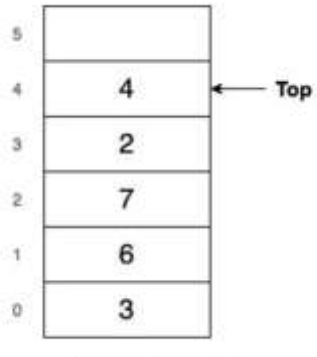
a) Stack of books b) stack of dishes

A stack is a special type of structure that can be used to maintain data in an organized way.

A stack is a list of elements in **which an element may be inserted or deleted only at one end**, called the top of the stack.

A stack is a LIFO (Last In First Out) data structure.

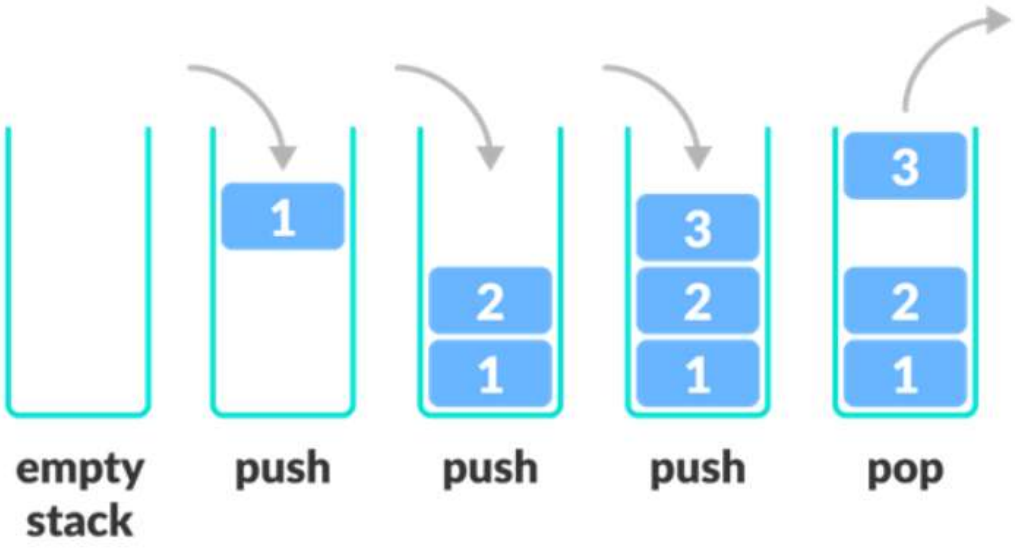
i.e. the elements are removed from a stack in the reverse order of that in which they were inserted into the stack.



Visualizing a Stack

There are ***two basic operations*** associated with stacks:

- **PUSH → is the term used to insert an element into a stack.**
- **POP → is the term used to delete an element from a stack.**



Practical Applications of Stack:

1. The UNDO functionality in text editors. The number of undos you can do is determined by the size of stack.
2. To keep track of the "most recently used" feature e.g. web browser uses the stack pattern to maintain the recently closed tabs and reopen them.
3. Code editors use a stack to check if you have closed all your parentheses properly or not.
4. Infix to postfix conversion
5. Evaluation of postfix expression.
6. Implementation of function calls and recursive functions.

Overflow & Underflow:

In executing the procedure PUSH, one must first test whether there is room in the stack for the new item; if not, then we have the condition known as overflow.

Analogously, in executing the procedure POP, one must first test whether there is an element in the stack to be deleted; if not, then we have the condition known as underflow.

overflow → no room for the new item in stack.

underflow → no element in the stack to be deleted.

A stack is commonly implemented in one of two ways:

1. **as an array or**
2. **as a linked list**

Array Representation of Stacks:

To implement a stack using array we will need:

- a linear **array STACK**,
- a **pointer variable TOP** → which contains the location of the top element of the stack and
- a **variable MAXSTK**→ which gives the maximum number of elements that can be held by the stack.

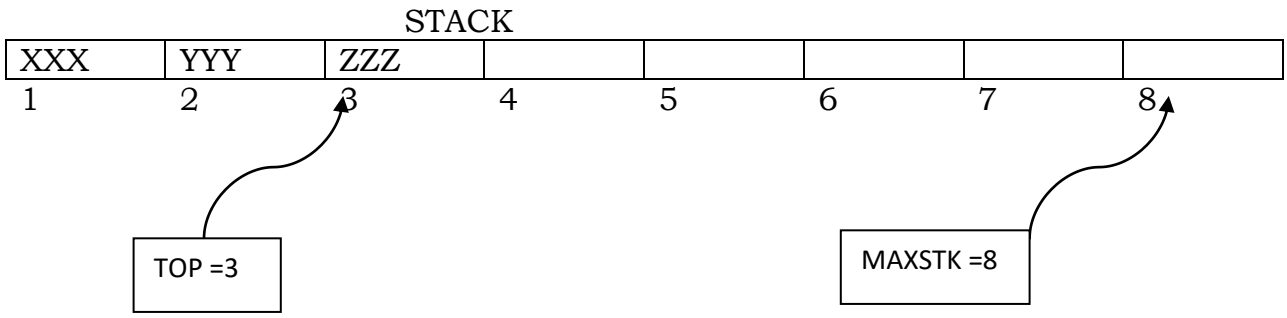


Fig above pictures an array representation of a stack. Since TOP =3, the stack has three elements: XXX, YYY, ZZZ and since MAXSTK =8, there is room for 5 more items in the stack.

Procedure for PUSH operation on stack:

PUSH (STACK, TOP, MAXSTK, ITEM)

This procedure pushes an ITEM onto a stack.

1. [Stack already filled?]
   If TOP = MAXSTK, then : Print: OVERFLOW, and Return.
2. Set TOP := TOP +1. [Increases TOP by 1.]
3. Set STACK[TOP] :=ITEM. [Inserts ITEM in new TOP position.]
4. Return

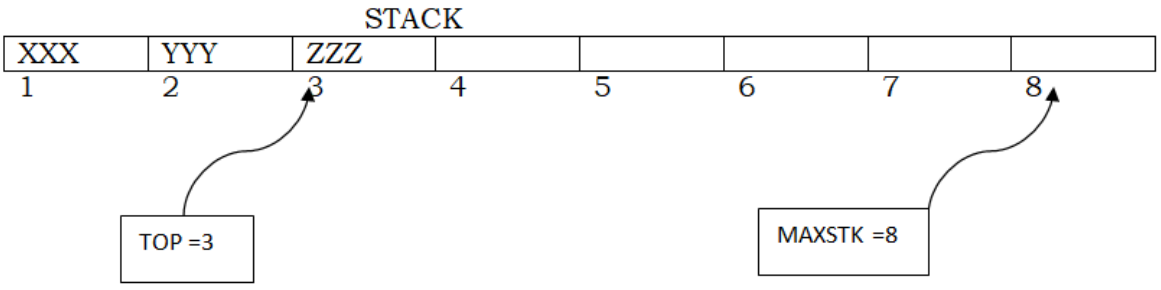
Procedure for POP operation on stack:

POP (STACK, TOP, ITEM)

This procedure deletes the top element of STACK and assigns it to the variable ITEM.

1. [Stack has an item to be removed?]
   If TOP = 0, then: Print: UNDERFLOW, and Return.
2. Set ITEM := STACK[TOP]. [Assigns TOP element to ITEM.]
3. Set TOP := TOP-1. [Decreases TOP by 1.]
4. Return.

Example:



Say in above stack, we want the operation PUSH (STACK, WWW)

1. Since TOP = 3, control is transferred to Step 2.
2. TOP =3+1 =4.
3. STACK[TOP] = STACK[4]=WWW.
4. Return.